Sistema de Gestión Hotelera

Alumna: Llanos, Jésica Débora

1.1 Descripción de la Temática

Introducción:

Se eligió realizar una base de datos de un hotel para aplicar los conocimientos aprendidos en MySQL Workbench. Esta base de datos permite llevar un mejor control de la información de los huéspedes, reservas, servicios solicitados y habitaciones de forma ordenada y más eficiente que los registros en papel.

Objetivo:

- Aplicar los conocimientos aprendidos para la creación de tablas y establecer sus correspondientes relaciones.
- Registrar y administrar la información de huéspedes, reservas, quejas y habitaciones.
- Facilitar la consulta y actualización de los datos almacenados en la base de datos.
- Permitir el análisis de la información obtenida y evaluar la calidad del servicio prestado.
- Permitir la implementación futura de vistas y permisos para mejorar la seguridad, la organización y la eficiencia de la base de datos.

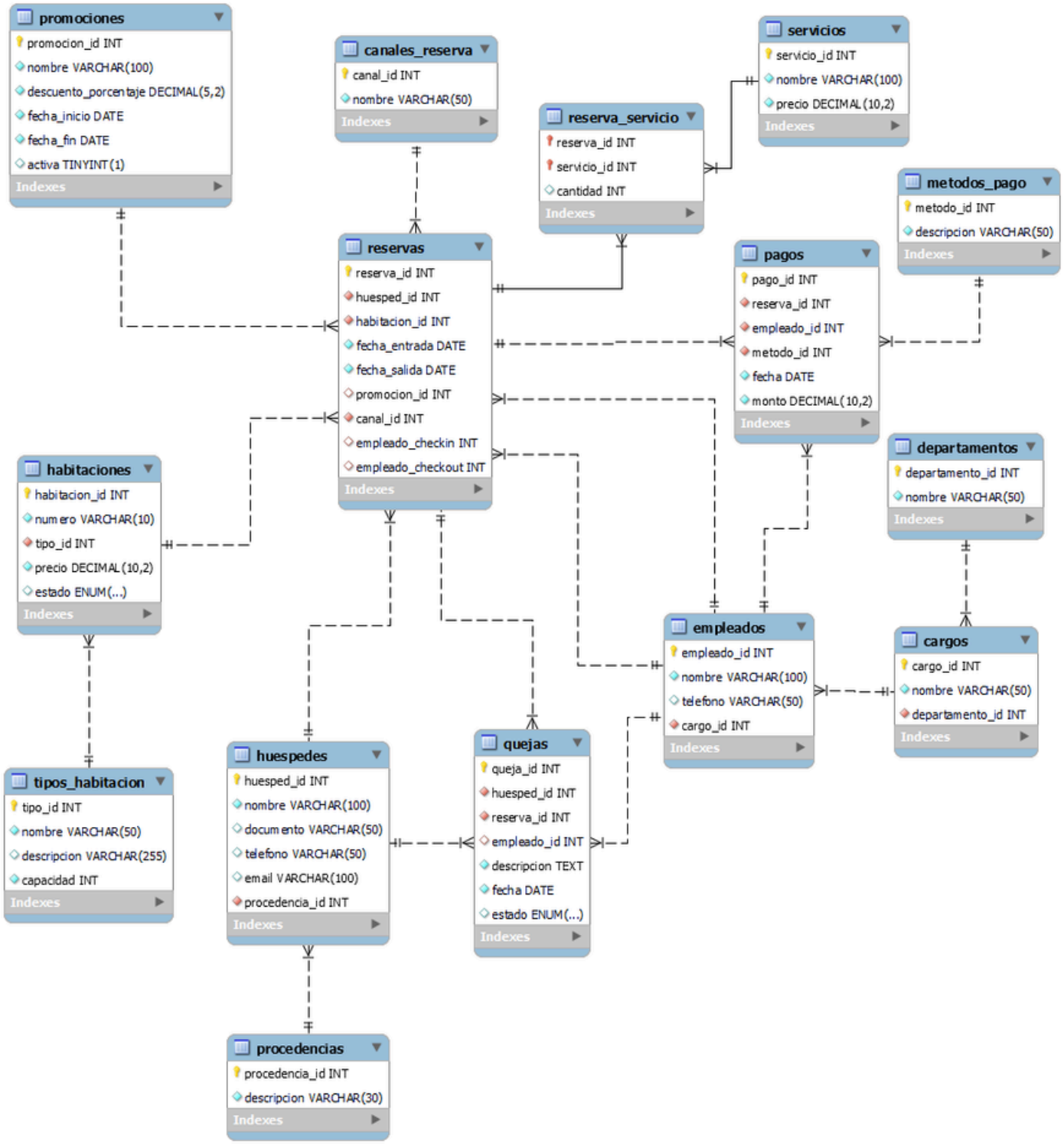
Situación Problemática:

Los registros físicos en libros de actas, cuadernos, papeles, o los archivos aislados en hojas de cálculo, hacen que la información del hotel sea vulnerable y se pueda perder o dañar, por ello se propone una **base de datos** que ayuda a organizar la información y evita que los datos se repitan o estén incorrectos. Además, permite hacer consultas rápidas y usar permisos y vistas para proteger la información y evitar errores.

Modelo de Negocio:

Mediante este sistema se logra centralizar la información operativa y financiera del hotel para facilitar el control, la toma de decisiones y el análisis histórico ya que permite saber quién se hospeda, procedencia nacional o internacional del huésped, en qué habitación lo hace, por cuánto tiempo, qué consume, cuánto paga, cómo paga, quién lo atiende y si tuvo algún problema.

1.2 Modelo Entidad Relación



Relaciones explicadas:

Procedencias → Huéspedes

Una procedencia puede tener muchos huéspedes

Un huésped tiene una sola procedencia

Huéspedes → Reservas

Un huésped puede hacer muchas reservas

Cada reserva es de un huésped

Tipos habitación → Habitaciones

Un tipo puede tener muchas habitaciones

Una habitación es de un tipo

Habitaciones → Reservas

Una habitación se usa en muchas reservas (en diferentes fechas)

Departamentos → Cargos → Empleados

Un departamento tiene muchos cargos

Un cargo pertenece a un departamento

Un empleado tiene un cargo

Reservas → Servicios (N:M)

Una reserva puede usar muchos servicios

Un servicio puede ser usado por muchas reservas

Se usa la tabla reserva_servicio para unirlos

Reservas → Pagos

Una reserva puede tener varios pagos

Empleados → Pagos

Un empleado cobra muchos pagos

Reservas → Quejas

Una reserva puede tener varias quejas

Huéspedes → Quejas

Un huésped puede hacer varias quejas

Empleados → Quejas

Un empleado puede estar relacionado con muchas quejas

1.3 Lista de Tablas

3.1 Tabla Procedencias:

La tabla procedencias tiene una PRIMARY KEY llamada procedencia_id. Esta tabla permitirá reconocer la procedencia nacional o internacional del huésped.

```
163 •  describe procedencias;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
procedencia_id	int	NO	PRI	NULL	auto_increment
descripcion	varchar(30)	NO	UNI	NULL	

3.2 Tabla Huéspedes:

La tabla huespedes tiene una clave primaria huesped_id y una clave foránea procedencia_id que referencia a la tabla procedencias. Ésta tabla sirve para guardar la información de cada persona que se hospeda.

```
170 •  describe huespedes;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
huesped_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	
documento	varchar(50)	YES		NULL	
telefono	varchar(50)	YES		NULL	
email	varchar(100)	YES		NULL	
procedencia_id	int	NO	MUL	NULL	

La columna procedencia_id es una clave foránea, por lo que MySQL le asigna automáticamente un índice no único (MUL).

3.3 Tabla tipos_habitacion

La tabla tipos_habitacion sirve para almacenar los tipos de habitaciones disponibles en el hotel, y su clave primaria es tipo_id.

```
171 •   describe tipos_habitacion;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

Field	Type	Null	Key	Default	Extra
tipo_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(50)	NO	UNI	NULL	
descripcion	varchar(255)	YES		NULL	
capacidad	int	NO		NULL	

3.4 Tabla habitaciones

La tabla habitaciones almacena los datos de las habitaciones del hotel.
Tiene como clave primaria habitacion_id, una clave foránea tipo_id y una clave única numero para que no se repita el número de habitación.

```
172 •   describe habitaciones;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
habitacion_id	int	NO	PRI	NULL	auto_increment
numero	varchar(10)	NO	UNI	NULL	
tipo_id	int	NO	MUL	NULL	
precio	decimal(10,2)	NO		NULL	
estado	enum('disponible','ocupada','mantenimiento')	YES		disponible	

3.5 Tabla departamentos

Ésta tabla sirve para guardar los distintos departamentos del hotel y su clave primaria es departamento_id. El campo nombre es **único**, o sea que no puede haber dos departamentos con el mismo nombre.

```
173 •   describe departamentos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
departamento_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(50)	NO	UNI	NULL	

3.6 Tabla cargos

Registra los distintos puestos de trabajo dentro de cada departamento

La tabla cargos tiene una clave primaria (cargo_id), una clave foránea (departamento_id) y una clave única compuesta (nombre, departamento_id); MUL aparece porque algunas columnas están indexadas pero no son únicas por sí solas.

```
174 •  describe cargos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
cargo_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(50)	NO	MUL	NULL	
departamento_id	int	NO	MUL	NULL	

3.7 Tabla empleados

Ésta tabla sirve para guardar la información de cada persona que trabaja en el hotel.

Su clave primaria es empleado_id y su clave foránea es cargo_id, que indica el cargo del empleado.

```
175 •  describe empleados;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
empleado_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	
telefono	varchar(50)	YES		NULL	
cargo_id	int	NO	MUL	NULL	

MUL aparece en cargo_id porque muchos empleados pueden tener el mismo cargo, y MySQL creó un índice para la relación con la tabla cargos.

3.8 Tabla Promociones

La tabla promociones sirve para guardar información sobre las ofertas o descuentos que ofrece el hotel. promocion_id es la clave primaria.

```
176 •  describe promociones;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
promocion_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	
descuento_porcentaje	decimal(5,2)	NO		NULL	
fecha_inicio	date	NO		NULL	
fecha_fin	date	NO		NULL	
activa	tinyint(1)	YES		1	

3.9 Tabla canales_reserva

Esta tabla registra los diferentes canales por los que alguien puede hacer una reserva (website,teléfono,App móvil,etc). Cada canal tiene un ID único (canal_id) y un nombre único (nombre).

```
177 •  describe canales_reserva;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
canal_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(50)	NO	UNI	NULL	

3.10 Tabla reservas

Esta tabla guarda toda la información de una reserva: quién la hizo, en qué habitación, por qué canal, si usó alguna promoción, y qué empleados registraron el check-in y check-out.

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

Field	Type	Null	Key	Default	Extra
reserva_id	int	NO	PRI	NULL	auto_increment
huesped_id	int	NO	MUL	NULL	
habitacion_id	int	NO	MUL	NULL	
fecha_entrada	date	NO		NULL	
fecha_salida	date	NO		NULL	
promocion_id	int	YES	MUL	NULL	
canal_id	int	NO	MUL	NULL	
empleado_checkin	int	YES	MUL	NULL	
empleado_checkout	int	YES	MUL	NULL	

Clave primaria: reserva_id

Claves foráneas:

- huesped_id → huespedes(huesped_id)
- habitacion_id → habitaciones(habitacion_id)
- promocion_id → promociones(promocion_id)
- canal_id → canales_reserva(canal_id)
- empleado_checkin → empleados(empleado_id)
- empleado_checkout → empleados(empleado_id)

3.11 Tabla Servicios

Esta tabla sirve para registrar los nombres de todos los servicios del hotel a los que los clientes pueden acceder, junto con su precio. Cada servicio tiene un ID único que lo identifica.

```
179 •  describe servicios;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
servicio_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	
precio	decimal(10,2)	NO		NULL	

Clave primaria: servicio_id

Clave foránea: ninguna

3.12 Tabla reserva_servicio

Esta tabla permite relacionar las reservas con los servicios que los clientes usan.Es una tabla intermedia entre reservas y servicios.

Cada fila indica qué servicio se usó en qué reserva y cuántas veces (cantidad).

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
reserva_id	int	NO	PRI	NULL	
servicio_id	int	NO	PRI	NULL	
cantidad	int	YES		1	

La tabla reserva_servicio tiene **dos claves foráneas**: reserva_id y servicio_id.

Juntas forman la **clave primaria compuesta** (reserva_id, servicio_id) para asegurar que no se repita la misma combinación.

3.13 Tabla metodos_pago

Esta tabla sirve para registrar todos los métodos de pago disponibles, de manera que cada método tenga un ID único y un nombre único.

```
181 •   describe metodos_pago;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cor

Field	Type	Null	Key	Default	Extra
metodo_id	int	NO	PRI	NULL	auto_increment
descripcion	varchar(50)	NO	UNI	NULL	

Clave primaria: metodo_id

Clave foránea: ninguna

3.14 Tabla pagos

La tabla pagos sirve para registrar todos los pagos que hacen los clientes.

	Field	Type	Null	Key	Default	Extra
▶	pago_id	int	NO	PRI	NULL	auto_increment
	reserva_id	int	NO	MUL	NULL	
	empleado_id	int	NO	MUL	NULL	
	metodo_id	int	NO	MUL	NULL	
	fecha	date	NO		NULL	
	monto	decimal(10,2)	NO		NULL	

Clave primaria: pago_id

Claves foráneas:

reserva_id → reservas(reserva_id)

empleado_id → empleados(empleado_id)

metodo_id → metodos_pago(metodo_id)

3.15 Tabla quejas

Esta tabla sirve para registrar todas las quejas o reclamos que hacen los huéspedes del hotel.

	Field	Type	Null	Key	Default	Extra
▶	queja_id	int	NO	PRI	NULL	auto_increment
	huesped_id	int	NO	MUL	NULL	
	reserva_id	int	NO	MUL	NULL	
	empleado_id	int	YES	MUL	NULL	
	descripcion	text	NO		NULL	
	fecha	date	NO		NULL	
	estado	enum('pendiente','en proceso','resuelta')	YES		pendiente	

Clave primaria: queja_id

Claves foráneas:

huesped_id → huespedes(huesped_id)

reserva_id → reservas(reserva_id)

empleado_id → empleados(empleado_id)

1.4 Link al archivo SQL

Links a los archivos correspondientes a esta primera entrega:

Github: https://github.com/Jesi10/hotel_regional

PDF:

https://drive.google.com/drive/folders/1lz-vGbEzNbY1L0gOLAuJS6aI9hpEEY9A?usp=sharing

El Script SQL llamado hotel_regional.sql permite la creación de la base de datos y la definición de sus tablas.

Uso:1. Abrir MySQL Workbench 2. Abrir `hotel_regional .sql` 3. Ejecutar el script ⚡

2.1 Listado de Vistas

1) Vista de servicios utilizados por reserva

Objetivo: Detallar qué servicios tomó cada huésped y el costo asociado.

Tablas involucradas: reserva_servicio, reservas, servicios, huespedes.

Beneficio: Mejora la precisión en cobros y control de ingresos extras.

reserva_id	huesped	servicio	cantidad	subtotal
4	Reyes Lozano Blanes	Excursion guiada	1	120.00
8	Desiderio Serrano Segovia	Excursion guiada	3	360.00
10	Pacfica Priego Francisco	Excursion guiada	1	120.00
12	Ani Querol Prieto	Excursion guiada	3	360.00
15	Isaura Gema Gisbert Nadal	Masaje	1	70.00
16	Maribel Nadal	Spa	1	50.00
19	Clotilde Espejo Sanmiguel	Clase de baile	2	60.00
20	Vasco Juregui	Spa	2	100.00
21	Nazaret Pera Chico	Excursion guiada	2	240.00

2) Vista de quejas NO resueltas

Objetivo: Listar todas las quejas que no han sido resueltas. Incluye las quejas con estado "pendiente" y "en proceso", excluyendo aquellas marcadas como "resuelta". Presenta además información relacionada con el huésped, la reserva y el empleado responsable.

Tablas involucradas: quejas, reservas, huespedes, empleados.

Beneficio: Permite al área de atención al cliente o gerencia dar seguimiento rápido y efectivo a las quejas de los huéspedes, mejorando la satisfacción de los huéspedes.

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

queja_id	huesped	reserva_id	descripcion	fecha	estado	empleado_responsable
1	Onofre Osuna Carpio	7	Inconveniente menor durante la estadÃ-a.	2025-01-02	en proceso	Carlos Gomez
2	Gilberto Pelez	17	Inconveniente menor durante la estadÃ-a.	2025-01-04	pendiente	Sofia Diaz
3	Clotilde Espejo Sanmiguel	27	Inconveniente menor durante la estadÃ-a.	2025-01-02	en proceso	Laura Martinez
4	Nazaret Pera Chico	30	Inconveniente menor durante la estadÃ-a.	2025-01-06	en proceso	Luciano Perez
5	Victor Sarmiento	32	Inconveniente menor durante la estadÃ-a.	2025-01-05	pendiente	Carlos Gomez
6	Lupe Via-Arce	40	Inconveniente menor durante la estadÃ-a.	2025-01-06	pendiente	Laura Martinez
7	Celso del Fuster	42	Inconveniente menor durante la estadÃ-a.	2025-01-05	en proceso	Sofia Diaz
8	Moiss Caete-Piol	44	Inconveniente menor durante la estadÃ-a.	2025-01-05	pendiente	Luciano Perez
9	Agapito Arce	76	Inconveniente menor durante la estadÃ-a.	2025-01-03	pendiente	Marina Lopez

3) Vista procedencias del año 2025

Objetivo:La vista vista_procedencias_top_2025 tiene como objetivo **identificar las principales procedencias geográficas de los huéspedes durante el año 2025**, mostrando cuántas reservas corresponden a cada procedencia y qué porcentaje representan sobre el total de reservas del mismo año.

Tablas involucradas:

reservas → contiene reserva_id, fecha_entrada, huesped_id

huespedes → contiene huesped_id, procedencia_id

procedencias → contiene procedencia_id, descripcion

Beneficio: Permite enfocar estrategias de marketing y promociones en las procedencias que más generan reservas y facilita la elaboración de reportes ejecutivos anuales.

procedencia_id	procedencia	total_reservas	porcentaje
9	Estados Unidos	1399	10.95
7	Francia	1333	10.43
8	Alemania	1326	10.38
5	Espana	1295	10.14
2	Chile	1294	10.13
6	Italia	1292	10.11
10	Canada	1276	9.99
3	Uruguay	1256	9.83
4	Brasil	1233	9.65

ista_procedencias_durante_20...

4) Vista historial de reserva por habitación

Objetivo: La vista vista_historial_reservas_habitaciones tiene como objetivo **centralizar la información histórica de las reservas asociadas a cada habitación del hotel**. Permite consultar en una sola estructura los datos de la habitación, su tipo, el huésped que realizó la reserva y las fechas de estadía, facilitando el análisis del uso de las habitaciones a lo largo del tiempo.

Tablas utilizadas:

habitaciones → número, tipo y estado de la habitación.

tipos_habitacion → nombre del tipo de habitación.

reservas → reserva asociada a la habitación, fechas de entrada y salida.

huespedes → nombre y documento del huésped que ocupa la habitación.

Beneficio: permite consultar el historial de ocupación de las habitaciones de forma rápida y centralizada, evitando realizar múltiples consultas sobre distintas tablas

habitacion	tipo_habitacion	estado	reserva_id	huesped	fecha_entrada	fecha_salida
101	Single	ocupada	13861	Otilia del Moya (61163281)	2026-02-06	2026-02-11
101	Single	ocupada	14056	Juliana Alfaro (94992582)	2026-02-12	2026-02-17
101	Single	ocupada	14212	Vasco Juregui (36794299)	2026-02-17	2026-02-19
101	Single	ocupada	14269	Lourdes Gmez Carranza (21315294)	2026-02-19	2026-02-20
101	Single	ocupada	14309	Severo Guijarro Villa (54525618)	2026-02-20	2026-02-23
101	Single	ocupada	14401	Isidro Sanchez Cabanillas (18415398)	2026-02-23	2026-02-28
102	Single	ocupada	2	Iker Santiago Roldan Anglada (87915002)	2025-01-01	2025-01-04
102	Single	ocupada	77	Guadalupe Boix Suarez (74357303)	2025-01-04	2025-01-09
102	Single	ocupada	218	Samanta Juliana Cantn Cabrera (293739...	2025-01-10	2025-01-12

sta_historial_reservas_habitaci... ×

5) Vista duración de la estadía por canal de reserva

Objetivo:
Analizar la duración promedio de estadía y el total de reservas por canal de reserva y por año.

Calcula para cada canal:

- cantidad de reservas realizadas
- duración promedio de la estadía
- año en que se realizaron las reservas

Tablas involucradas:

- reservas → fecha_entrada, fecha_salida, reserva_id,
- canales_reserva → nombre del canal, canal_id

Beneficio: Esta vista permite analizar el rendimiento de cada canal de reservas, mostrando la cantidad de reservas realizadas y la duración promedio de estadía por año. Cabe aclarar que los datos actualmente cargados en el sistema corresponden al año 2025 completo y al año 2026 únicamente hasta la fecha.

canal_id	canal	anio	total_reservas	duracion_promedio
5	Agencia de Viajes	2025	2498	2.97
5	Agencia de Viajes	2026	361	2.99
2	Booking	2025	2490	2.97
2	Booking	2026	379	3.02
3	Recepción Presencial	2025	2533	3.00
3	Recepción Presencial	2026	338	2.91
1	Sitio Web	2025	2604	2.97
1	Sitio Web	2026	374	2.92
4	Teléfono	2025	2652	3.03

2.2 Listado de Funciones

1) Función calcular_noches

Objetivo: La función permite determinar cuán larga es la estadía de un huésped calculando la cantidad de noches entre la fecha de entrada y la fecha de salida de una reserva. Utiliza la función interna de MySQL DATEDIFF para obtener la diferencia de días entre ambas fechas.

Tablas involucradas: La función no depende directamente de una tabla aplica principalmente sobre los campos fecha_entrada fecha_salida de la tabla reservas.

Beneficios: Análisis estadísticos, reportes de ocupación. Evita repetir cálculos en múltiples consultas.

2) Función total_consumo

Objetivo: Calcular el importe total gastado por un huésped en los servicios contratados durante sus reservas.

La función recibe como parámetro un valor entero (INT) correspondiente al **huesped_id** y devuelve un valor numérico de tipo **DECIMAL(10,2)**, que representa el total monetario consumido en servicios.

Internamente, la función:

1. Declara una variable local llamada total.
2. Realiza una consulta que: Suma el resultado de multiplicar el precio del servicio (s.precio) por la cantidad contratada (rs.cantidad).
3. Relaciona múltiples tablas mediante JOIN para obtener la información necesaria.
4. Filtra los resultados según el huésped recibido como parámetro.
5. Devuelve el total calculado.
6. En caso de que el huésped no tenga consumos registrados, devuelve 0 mediante el uso de IFNULL.

La función está declarada como DETERMINISTIC, ya que para un mismo huesped_id devolverá siempre el mismo resultado mientras los datos no cambien.

Tablas involucradas: reserva_servicio ,reservas y servicios

Beneficio de su implementación en el hotel: Identificar clientes con mayor consumo. Permite centralizar el cálculo del gasto total de un huésped.

2.3 Stored Procedures

1) Stored procedure sp_calcular_estado_reserva_final

Objetivo: calcular la **cuenta completa** de una reserva del hotel.Es decir: cuánto costó la habitación,cuánto se gastó en servicios,si hay promociones,cuánto pagó el huésped,si todavía debe dinero o pagó de más. Generar un resumen de facturación automático.

Este SP recibe como parámetro de entrada el identificador de la reserva (p_reserva_id).

A partir de ese identificador, el procedimiento realiza los siguientes pasos:

- Obtiene el precio de la habitación y las fechas de la reserva.
- Calcula la cantidad de noches de hospedaje.
- Calcula el total del hospedaje.
- Suma los servicios consumidos durante la estadía.
- Verifica si existe una promoción activa.
- Suma todos los pagos realizados.
- Verifica si se utilizó transferencia bancaria como método de pago.
- Aplica el descuento correspondiente.
- Calcula el total final de la reserva.
- Determina el estado financiero de la reserva: Deuda pendiente, Reserva paga, Saldo a favor del huésped.

Tablas que intervienen: reservas, habitaciones, reserva_servicio, servicios, promociones, pagos, metodos_pago

Beneficios para el hotel: Permite conocer rápidamente el estado de cada reserva,detectar deudas pendientes,detectar pagos excesivos,controlar promociones, evitar errores manuales en facturación, generar reportes administrativos

Ejemplos:

Reserva Paga

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Reserva	Noches	Precio_por_noche	Total_Habitacion	Servicios	Descuento	Origen_Descuento	Pagos	Total_Final	Estado
1	2	80.00	160.00	0.00	16.00	Promo Verano 2026 (10.00%)	144.00	144.00	Reserva paga

Deuda Pendiente

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Reserva	Noches	Precio_por_noche	Total_Habitacion	Servicios	Descuento	Origen_Descuento	Pagos	Total_Final	Estado
22	3	80.00	240.00	0.00	24.00	Promo Verano 2026 (10.00%)	136.00	216.00	Deuda pendiente: $80.00

Saldo a favor del huésped

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Reserva	Noches	Precio_por_noche	Total_Habitacion	Servicios	Descuento	Origen_Descuento	Pagos	Total_Final	Estado
1000	5	160.00	800.00	90.00	80.00	Descuento 10% por transferencia	1400.00	810.00	Saldo a favor del huésped: $590.00

2) Stored procedure sp_buscar_habitaciones_disponibles

Objetivo: Identificar las habitaciones que se encuentran libres en un rango de fechas específico para facilitar la gestión de reservas mostrando habitaciones disponibles y proporcionando información clara al usuario o al sistema sobre la disponibilidad en el período indicado.

Recibe como parámetros:

- **IN** p_fecha_inicio (DATE): fecha de inicio de la reserva buscada.
- **IN** p_fecha_fin (DATE): fecha de fin de la reserva buscada.
- **OUT** p_mensaje (VARCHAR(200)): mensaje que indica si hay disponibilidad y cuántas habitaciones están libres.

Tablas involucradas: Interactúa con las tablas habitaciones, tipos_habitacion y reservas, mostrando las habitaciones libres, contabilizando los resultados obtenidos y devolviendo mediante el parámetro de salida p_mensaje un mensaje informativo sobre la disponibilidad encontrada.

Beneficios: Mejora la gestión operativa del hotel porque permite automatizar la verificación de disponibilidad de habitaciones, evitando reservas superpuestas.

```
3  CALL sp_buscar_habitaciones_disponibles(
4  '2030-01-01',
5  '2030-01-10',
6  @mensaje
7  );
8  SELECT @mensaje;
```

Result Grid | Filter Rows: | Export:

@mensaje
Busqueda exitosa. Habitaciones disponibles: 5

2.4 Triggers

Se implementaron los siguientes triggers:

1)Trigger trg_before_insert_reserva_definitivo

Objetivos:Evitar errores y conflictos al crear reservas en el hotel. Garantizar que:

- La fecha de salida sea posterior a la fecha de entrada.
- La habitación no esté en mantenimiento.
- La habitación no esté ocupada.
- No haya reservas superpuestas para la misma habitación.

Funcionamiento:

Se ejecuta antes de insertar la reserva, lo que permite bloquear la operación si algo falla.

Usa NEW para acceder a los datos de la reserva que se intenta crear (fecha_entrada, fecha_salida, habitacion_id).

Consulta la tabla *habitaciones* para obtener el estado de la habitación.

Verifica condiciones de:1)Fechas, 2)Estado (mantenimiento / ocupada), 3)Superposición con otras reservas (reserva_id y fechas existentes)

Si alguna condición falla, lanza un error (SIGNAL SQLSTATE '45000') y detiene la inserción.

Tablas involucradas: reservas y habitaciones

Beneficio de su aplicación: No se crean reservas inválidas ni sobrepuestas respetando la Integridad de la base de datos.Asegura que no haya dos huéspedes para la misma habitación al mismo tiempo. Evita de esta manera conflictos y errores al momento del check-in.

2)Trigger trg_after_insert_reserva_actualizar_habitacion

Es un ejemplo de automatización posterior, ya que luego de registrarse una reserva se ejecuta automáticamente el trigger, actualizando el estado de la habitación asociada.

Está definido sobre la tabla reservas y se ejecuta después de insertar un nuevo registro (AFTER INSERT). Su función es actualizar automáticamente el estado de la habitación asociada a la reserva, cambiándolo a **ocupada** cuando se registra una nueva reserva.

El objetivo de este trigger es evitar que una habitación reservada continúe figurando como disponible dentro del sistema.

Para ello, el trigger toma el identificador de la habitación almacenado en el nuevo registro insertado y actualiza su estado en la tabla habitaciones.

Intervienen las tablas reservas (tabla que dispara el trigger) y habitaciones (tabla afectada por la actualización del estado).

2.5 Archivos SQL y CSV

En el siguiente enlace se puede acceder al drive que contiene los archivos correspondientes a la segunda entrega.

https://drive.google.com/drive/folders/1p8MyEjMKolOFgJjxGV4RA8cPZYfTdaUn?usp=sharing

Descripción de archivos

01_Creacion_Vistas_SP_Triggers_Funciones.sql: Contiene la creación de objetos: Vistas, Funciones, Stored Procedures, Triggers. Se debe ejecutar antes de cargar los datos, para que los objetos funcionen correctamente.

02_Insercion_datos.sql: Contiene las sentencias INSERT INTO para poblar las tablas independientes (tablas padre) con los datos iniciales. Se ejecuta después de crear los objetos, garantizando que triggers y procedimientos puedan aplicarse correctamente.

Carpeta csv_archivos: mediante el asistente gráfico Table Data Import Wizard de MySQL Workbench se debe realizar la importación de archivos en el siguiente orden:

1. procedencias.csv
2. habitaciones.csv
3. servicios.csv
4. huespedes.csv
5. reservas.csv
6. reserva_servicio.csv
7. pagos.csv
8. quejas.csv

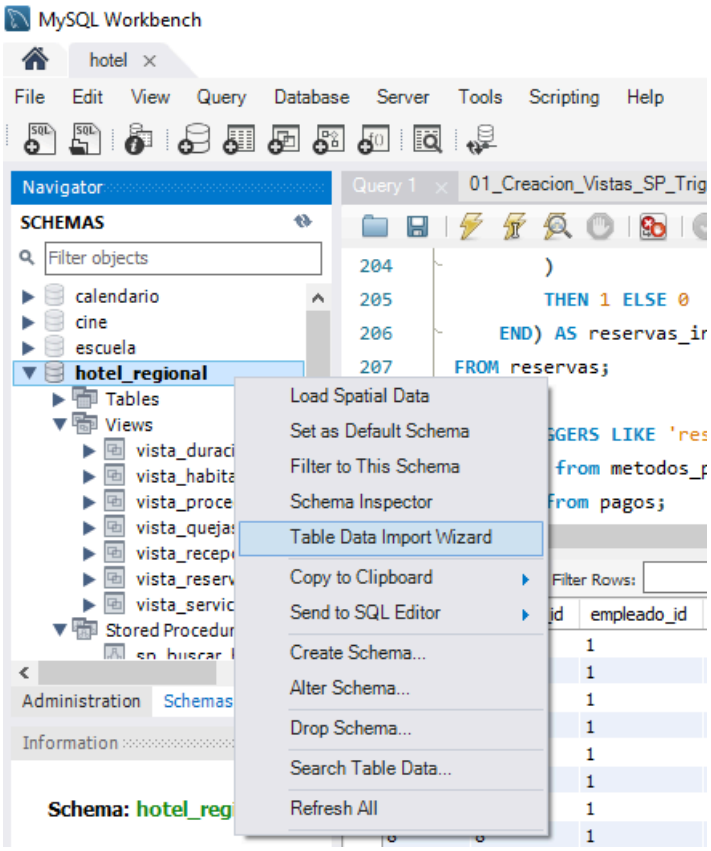
Respetar el orden de carga de los archivos .csv para evitar conflictos con llaves foráneas.

Importación de datos mediante archivos .csv

A continuación, se detalla el proceso de inserción de datos utilizando el asistente gráfico.

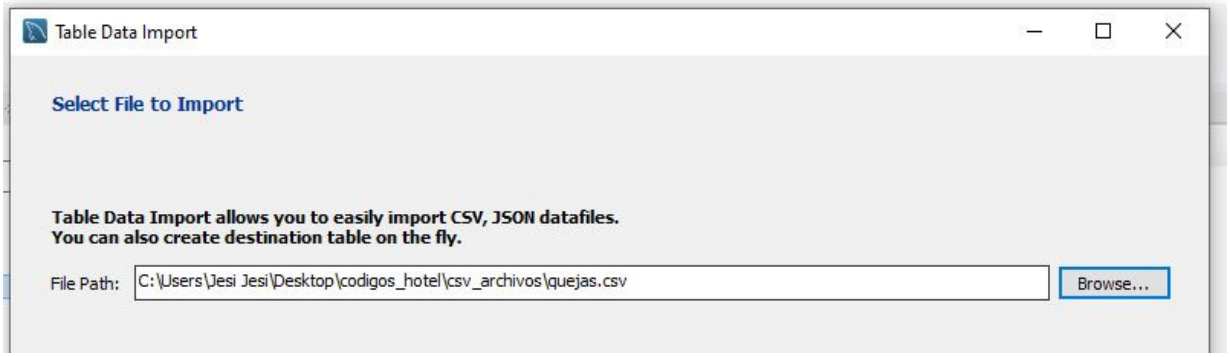
1.Seleccionar la base de datos hotel_regional

2.Iniciar el asistente: Hacer click derecho sobre la base de datos (hotel_regional) y se despliega un menú. Entre las opciones disponibles seleccionar ***Table Data Import Wizard***. Éste es el asistente gráfico de Workbench.



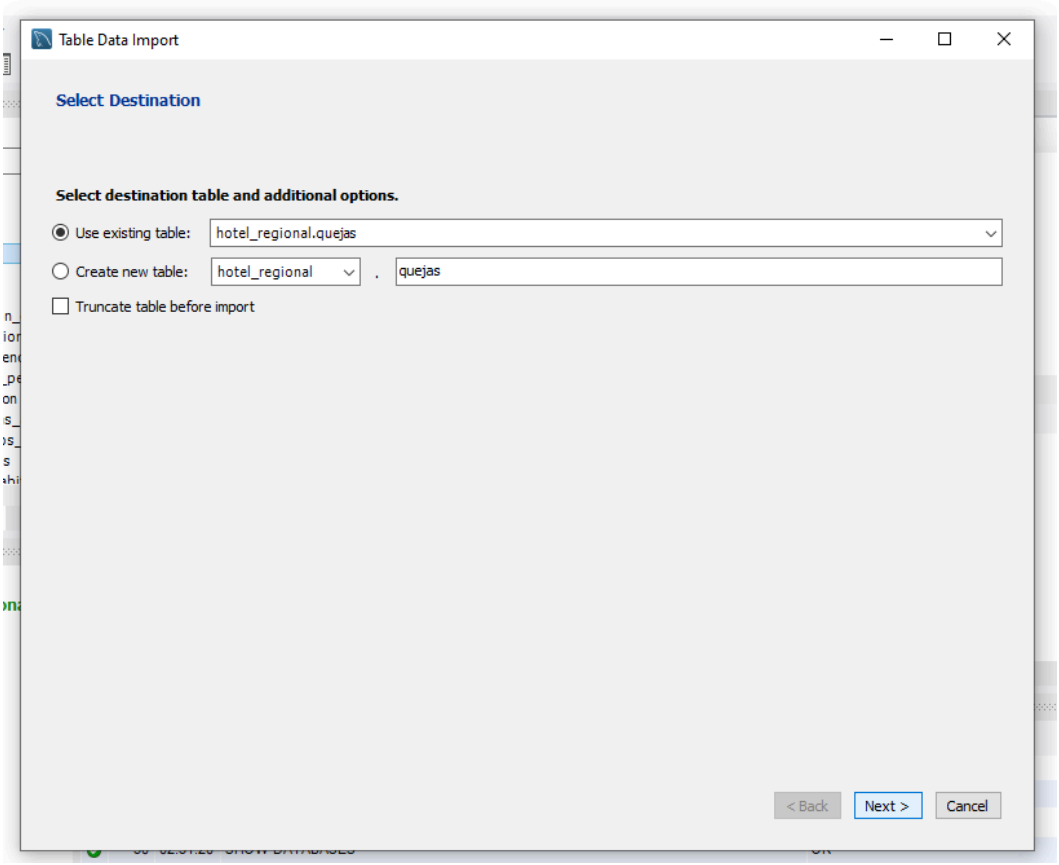
3.Elegir el archivo CSV

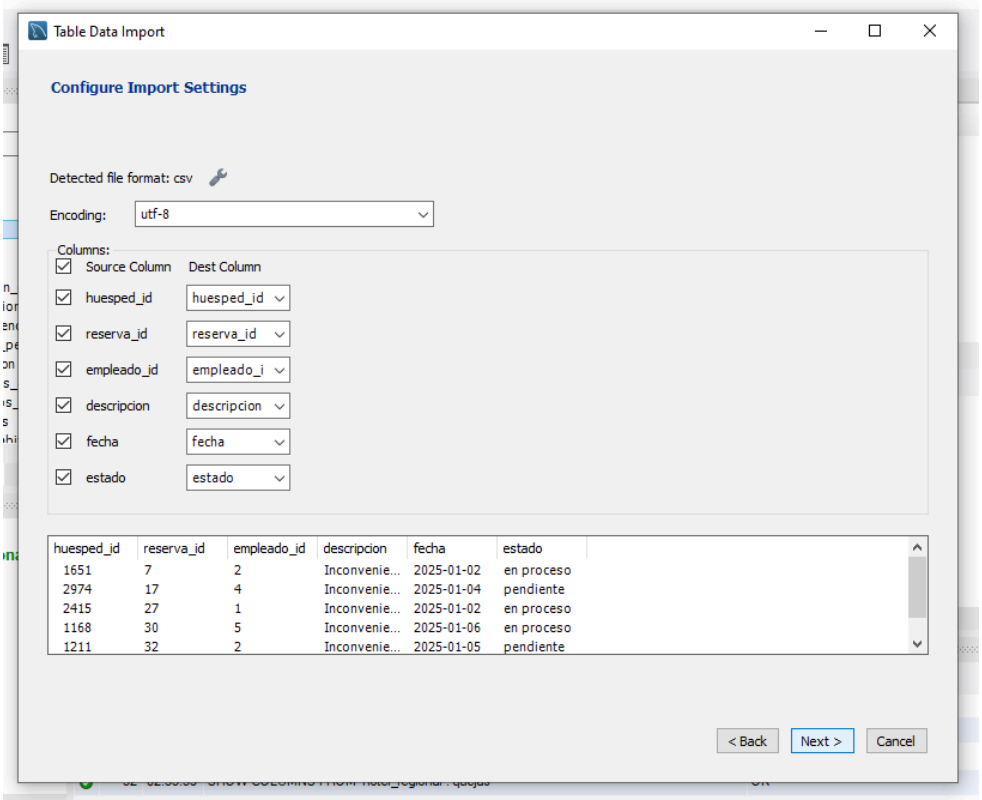
a. Browse → seleccionar archivo → **Next**.



En la imagen se seleccionó el archivo llamado quejas.csv

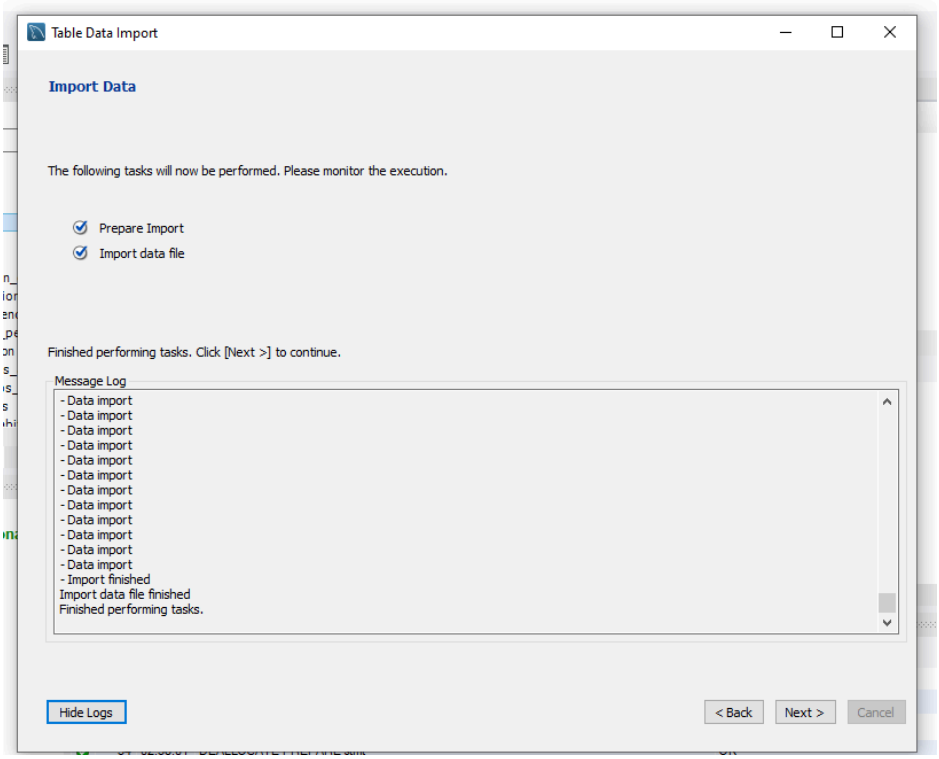
4.Configurar tabla : Se seleccionó la opción usar tabla existente porque ya esta creada la tabla de destino de los datos. Luego se revisaron nombres y tipos de columnas → **Next**.

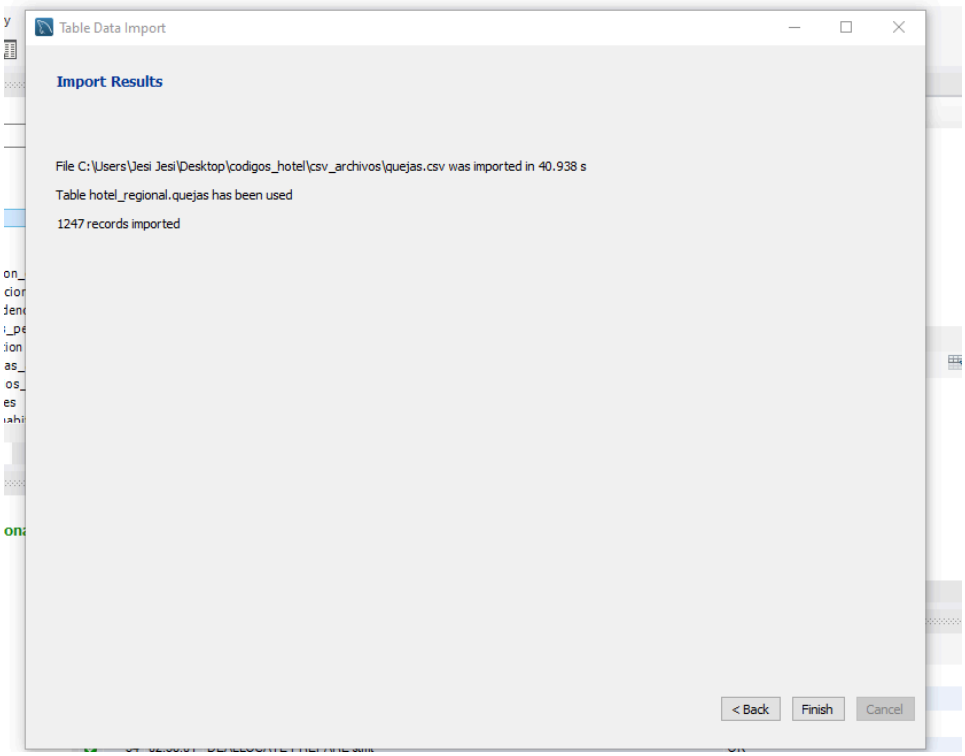




*Revisamos que los campos coincidan y luego **Next***

5.Finalizar importación: Revisar resumen → **Finish**.





6.Repetir estos pasos para los demás CSV